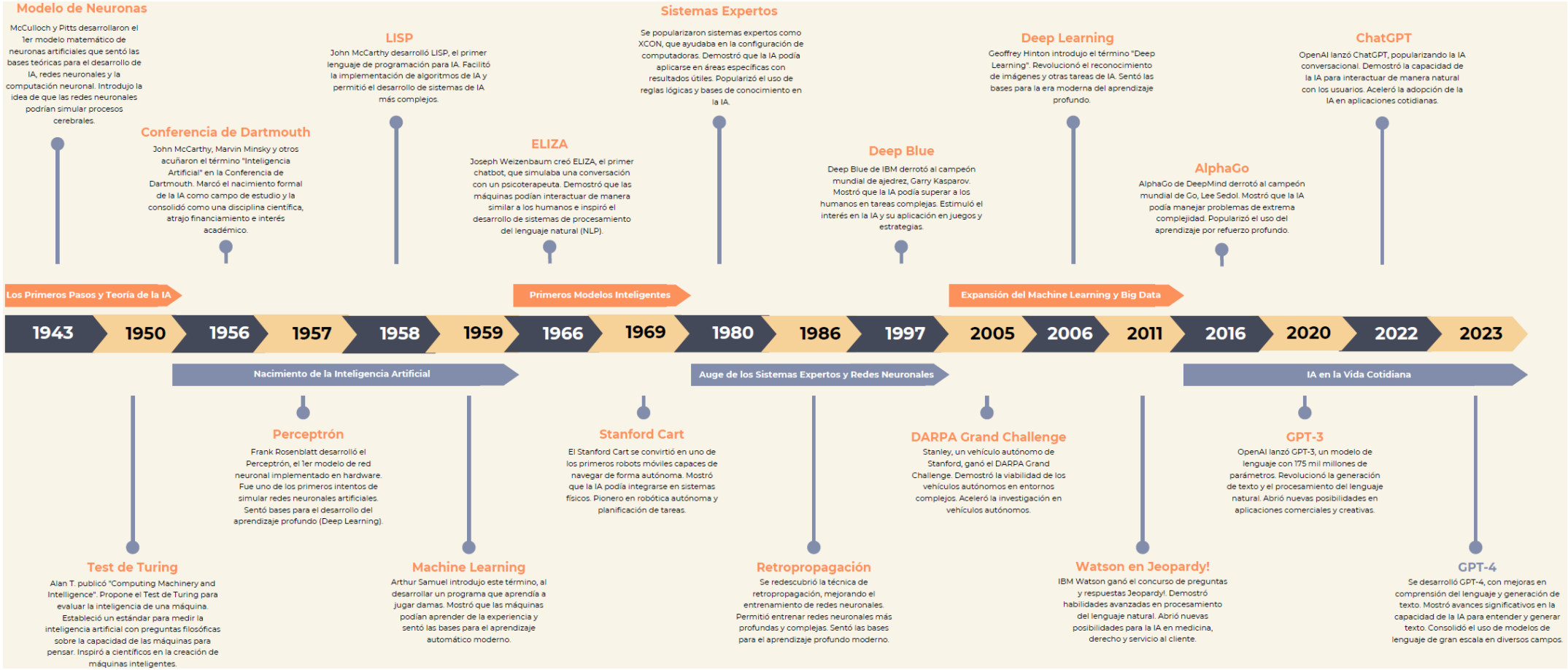
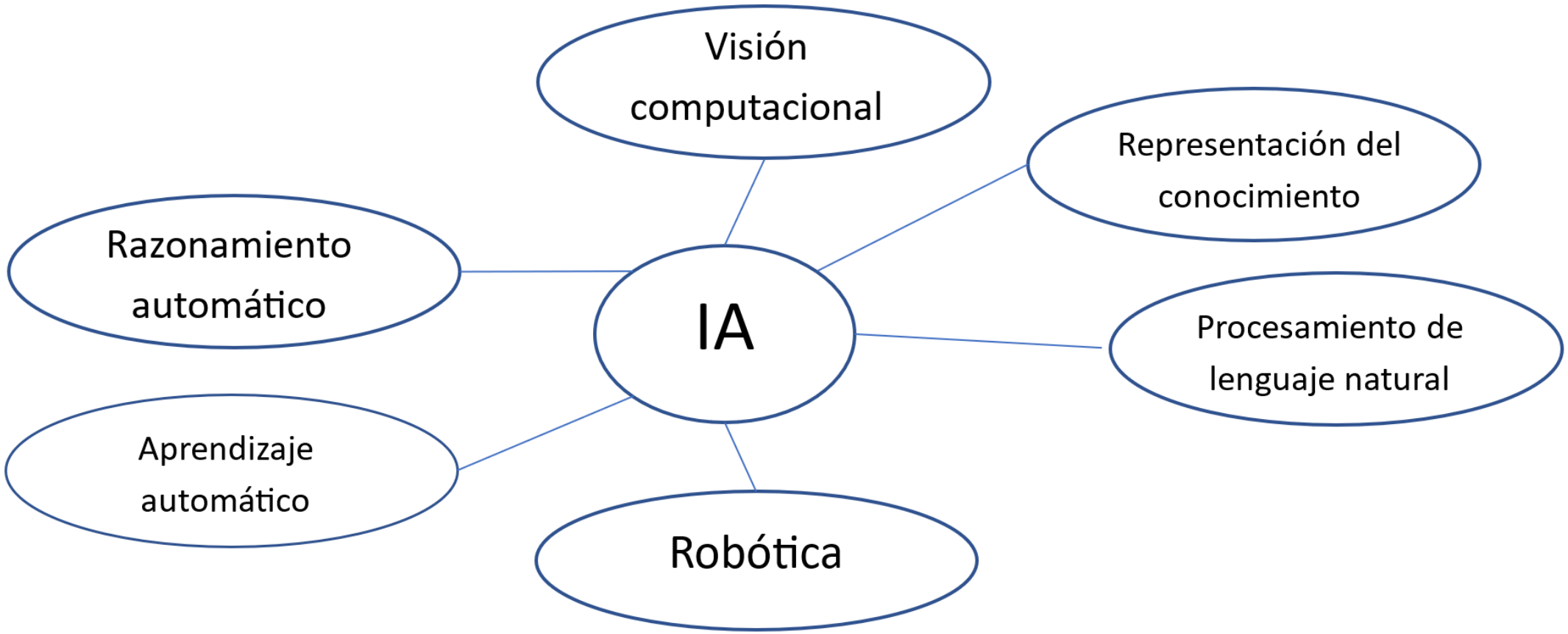
Inteligencia artificial

Unidad 1.- introducción a la inteligencia artificial

1. Historia de la inteligencia artificial



2. Áreas de la IA y su campo de aplicación



Aprendizaje automático

Machine learning o aprendizaje automático, se divide en distintos tipos de algoritmos, entre ellos los algoritmos de agrupación.

Esta disciplina, enmarcada en la inteligencia artificial, crea distintos sistemas que permite a las máquinas aprender de forma automática. En este sentido, consigue que los ordenadores lleven a cabo tareas de forma independiente.

Dentro del aprendizaje automático encontramos tres grupos de algoritmos:

- **Aprendizaje supervisado**
- **Aprendizaje no supervisado**
- **Aprendizaje reforzado**

Aprendizaje supervisado

El modelo aprende con datos etiquetados (entrada → salida conocida).

Tareas:

Clasificación

Regresión

Aprendizaje no supervisado

El modelo encuentra patrones sin etiquetas.

Tareas:

Clustering

Reducción de dimensionalidad

Aprendizaje por refuerzo

El modelo aprende mediante prueba y error con recompensas.

Ejemplo:

Un robot que aprende a moverse sin chocar.

A su vez, estos 3 tipos algoritmos de se dividen en distintos tipos, entre los que encontramos:

- Algoritmos de regresión
- Algoritmos bayesianos
- **Algoritmos de agrupación**
- Algoritmos de árbol de decisión
- Algoritmos de reducción de dimensión
- Algoritmos Deep learning
- Otros

Aprendizaje supervisado

El modelo aprende con datos etiquetados (entrada → salida conocida).

Tareas:

Clasificación

Regresión



Aprendizaje no supervisado

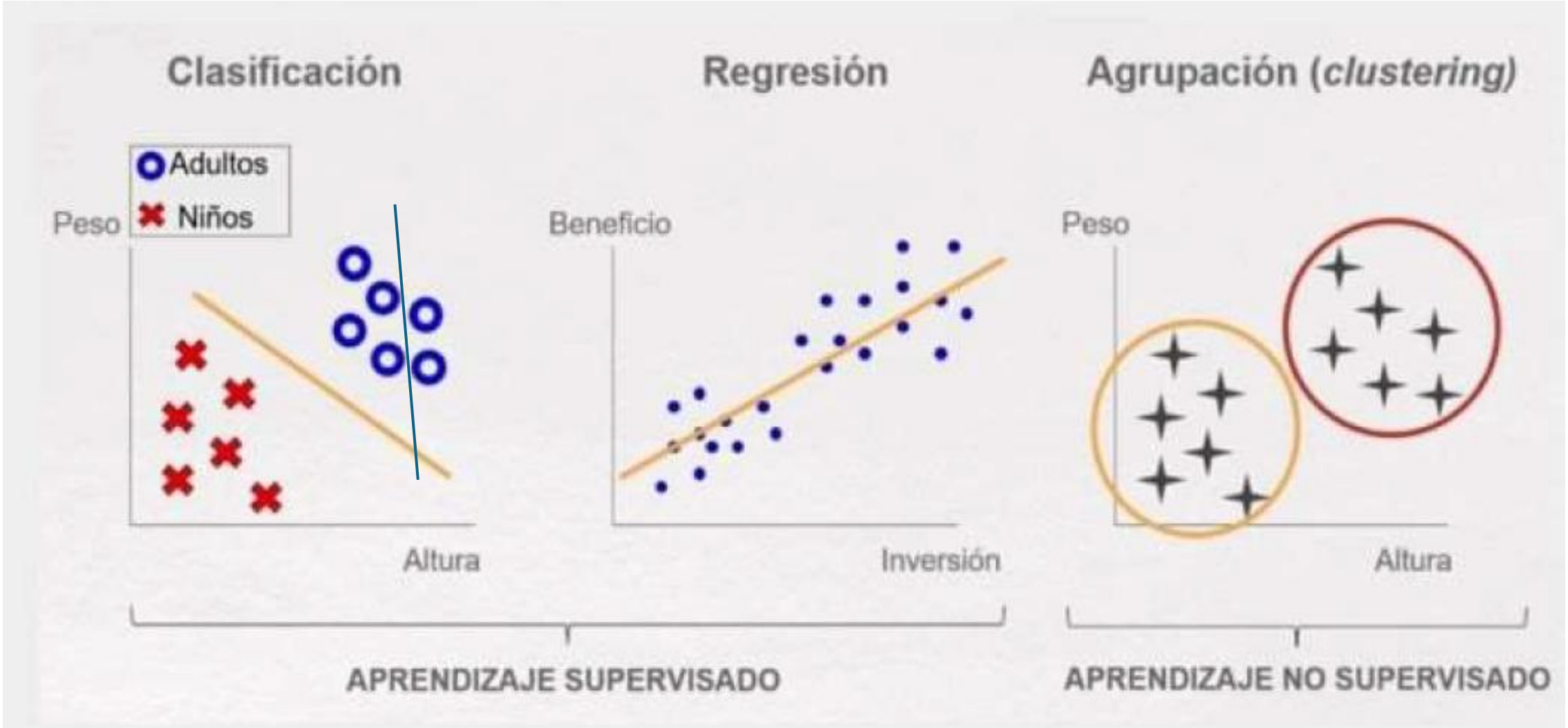
El modelo encuentra patrones sin etiquetas.

Tareas:

Clustering

Reducción de dimensionalidad




Clasificación
Regresión
Agrupación (*clustering*)
Adultos
Niños
Peso
Altura
Beneficio
Inversión
Peso
Altura
APRENDIZAJE SUPERVISADO
APRENDIZAJE NO SUPERVISADO

¿Qué es un algoritmo de agrupación?

- Dentro de los algoritmos de aprendizaje no supervisado, nos encontramos con los algoritmos de agrupación. **Se trata de una técnica común en el análisis de datos estadísticos y que se utiliza a menudo en diversos campos.**
- Su utilidad es, principalmente, **categorizar datos que no hayan sido previamente etiquetados. Es decir, que no tengan ni categorías ni grupos definidos.**
- El funcionamiento de los algoritmos de agrupación **se basa en la búsqueda de grupos dentro de los datos, con un número de grupos que se representa a través de la letra K.** Tras esto, procede a asignar cada punto de datos a uno de los K grupos, dependiendo de las características que se le hayan proporcionado.

3. Métodos de agrupamiento

- Un método de agrupamiento es un algoritmo de aprendizaje automático no supervisado, en el que se identifican patrones dentro de un conjunto de datos y el método genera automáticamente subgrupos de tipos similares de variables de entrada, también conocidos como clústeres .
- Según esta definición, los algoritmos de agrupamiento generalmente operan con el supuesto de que existe un patrón subyacente en el conjunto de datos de entrada. Como resultado, sus subgrupos generados deben ser validados estadística y científicamente.
- Los algoritmos de agrupamiento dependen en gran medida de cómo se define la similitud (ver medidas de similitud) entre los objetos en un conjunto de datos. En el concepto de procesamiento de nubes de puntos, el agrupamiento se refiere a agrupar puntos en función de su distancia euclidiana de otros puntos y características como los valores RGB de cada punto y las normales y curvaturas estimadas.
- Dos de los algoritmos de agrupamiento más comunes para el procesamiento de nubes de puntos son el agrupamiento jerárquico y los algoritmos k-means.

Características de los algoritmos de agrupación

Algunas de las características principales de este tipo de algoritmo son las siguientes:

- Los datos que utilizan no tienen ninguna etiqueta previa
- El algoritmo tiene la capacidad de encontrar todos los datos sin supervisión
- Se trata de un algoritmo esencial para que los científicos de datos descubran agrupaciones

Tipos de algoritmos de agrupación

A su vez, los algoritmos de agrupación se pueden dividir en distintos tipos de algoritmo, con la capacidad de manejar todo tipo de datos no etiquetados.

- **Basados en la densidad:** en él, los datos se agrupan según las áreas de altas concentraciones de datos que se rodean de áreas bajas de concentración de puntos de datos. A estos lugares los llama grupos.
- **Algoritmos de distribución:** se basa en la distribución y funciona detectando un punto central de datos que, a medida que aumenta la distancia a este desde el centro, la posibilidad de que se haga parte del grupo, decrece.
- **Algoritmos basados en centroides: es el más común, rápido y eficiente**. Su función es separar puntos de datos, asignando cada uno de ellos a la distancia al cuadrado del centroide.
- **Basados en jerarquías:** su utilidad está, principalmente, destinada a los datos jerárquicos. **Construye un árbol de grupos para que todo esté organizado de arriba hacia abajo.**

Cluster K-Means: el algoritmo de agrupación

- Se trata de un algoritmo **perteneciente al grupo de algoritmos basados en centroides** y es el algoritmo de aprendizaje no supervisado más simple, pero también el más veloz**.**
- **Su función es minimizar la variedad de puntos dentro de un grupo y se usa, preferiblemente,** en conjuntos de datos más

¿Cuándo se utilizan los algoritmos de agrupación?

Existen distintas situaciones reales en las que se usan los algoritmos de agrupación. Algunas de ellas son:

- Ventas y marketing
- Identificación de criminales
- Detección de noticias falsas
- Filtros de *spam*

Definición formal de agrupamiento

El **agrupamiento** (*clustering*) es una técnica de **aprendizaje no supervisado** cuyo objetivo es particionar un conjunto de observaciones en grupos o *clusters*, de manera que los elementos pertenecientes a un mismo grupo presenten una mayor similitud entre sí que respecto de los elementos de otros grupos.

Formalmente, dado un conjunto de datos:

$$X = \{x_1, x_2, \ldots, x_n\}$$

donde cada observación x_i está representada mediante un conjunto de características:

$$x_i = (x_{i1}, x_{i2}, ..., x_{ip})$$

el objetivo es encontrar una partición:

$$C = \{C_1, C_2, \ldots, C_k\}$$

que maximice la similitud intra-cluster y minimice la similitud inter-cluster.

A diferencia del aprendizaje supervisado, **no se dispone de una variable objetivo previamente etiquetada**.

Medidas de similitud y distancia

Los algoritmos de agrupamiento requieren una función que permita determinar qué tan cercanos o similares son dos objetos.

Una de las más utilizadas es la **distancia euclidiana**:

$$d(x, y) = \sqrt{\sum_{j=1}^{p} (x_j - y_j)^2}$$

También pueden utilizarse:

- distancia Manhattan
- distancia Minkowski
- distancia de Mahalanobis
- similitud coseno
- distancia de Jaccard

La elección de la métrica depende de la naturaleza de los datos.

Por ejemplo, en documentos representados mediante vectores TF-IDF suele ser más apropiada la **similitud coseno** que la distancia euclidiana.

K-means

K-means es uno de los algoritmos de agrupamiento particional más utilizados.

Se establece previamente el número de clusters k.

El algoritmo busca los centroides:

$$\mu_1, \mu_2, \ldots, \mu_k$$

que minimizan la suma de las distancias cuadráticas entre cada observación y el centroide de su cluster.

La función objetivo es:

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Algoritmo

1. Seleccionar k centroides iniciales.
2. Asignar cada observación al centroide más cercano.
3. Recalcular los centroides.
4. Repetir los pasos 2 y 3 hasta alcanzar convergencia.

K-means

Ventajas

- Simple de implementar.
- Buena eficiencia computacional.
- Escalable para conjuntos relativamente grandes.
- Fácil de interpretar.

Limitaciones

- Requiere determinar k.
- Sensible a la inicialización.
- Sensible a valores atípicos.
- Funciona mejor cuando los clusters presentan formas aproximadamente convexas o esféricas.
- La escala de las variables puede afectar considerablemente el resultado.

Por ello, antes de aplicar K-means suele ser necesario analizar la **normalización o estandarización de variables**.

K-means++

- Una mejora importante es **K-means++**, que modifica la selección inicial de centroides para reducir la probabilidad de obtener una solución deficiente debido a una mala inicialización.

Por ello, en implementaciones prácticas suele preferirse:

```
KMeans(n_clusters=k, init="k-means++")
```

frente a una inicialización completamente aleatoria.

Agrupamiento jerárquico

El **clustering jerárquico** construye una estructura jerárquica de los datos.

Existen dos estrategias principales:

Aglomerativa

Comienza considerando cada observación como un cluster independiente:

$$\{x_1\}, \{x_2\}, ..., \{x_n\}$$

y sucesivamente fusiona los clusters más próximos.

Divisiva

Comienza con un único cluster que contiene todos los datos y lo divide progresivamente.

La estrategia aglomerativa es la más utilizada.

El resultado puede representarse mediante un **dendrograma**.

Métodos de enlace

Para determinar la distancia entre clusters pueden utilizarse:

- **Single linkage**
- **Complete linkage**
- **Average linkage**
- **Ward linkage**

El método de Ward, por ejemplo, intenta minimizar el incremento de la varianza interna producido por cada fusión.

DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) es un algoritmo basado en densidad.

A diferencia de K-means, no requiere especificar directamente el número de clusters.

Utiliza dos parámetros:

$$\epsilon$$

radio de vecindad, y

$$MinPts$$

número mínimo de puntos necesarios para considerar que existe una región densa.

Los puntos pueden clasificarse como:

- **core points**
- **border points**
- **noise points**

Una ventaja importante de DBSCAN es que puede detectar clusters de formas arbitrarias y distinguir valores atípicos.

Evaluación de agrupamientos

Un problema fundamental es determinar si los clusters obtenidos son adecuados.

Entre las métricas utilizadas están:

Silhouette Score

Evalúa simultáneamente cohesión y separación:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

donde:

- $a(i)$ = distancia promedio del punto respecto de los elementos de su propio cluster.
- $b(i)$ = distancia promedio respecto del cluster vecino más cercano.

El valor se encuentra aproximadamente entre:

$$-1 \leq s(i) \leq 1$$

Valores cercanos a 1 indican una buena asignación.

También pueden utilizarse:

- Calinski-Harabasz Index
- Davies-Bouldin Index
- Adjusted Rand Index, cuando existen etiquetas reales
- Normalized Mutual Information

Aplicaciones

El clustering tiene aplicaciones en:

- segmentación de clientes
- análisis de perfiles académicos
- minería de texto
- agrupamiento de documentos
- bioinformática
- reconocimiento de patrones
- segmentación de imágenes
- detección de anomalías
- análisis exploratorio de datos

En **Procesamiento de Lenguaje Natural**, por ejemplo, puede utilizarse para agrupar documentos por temática sin disponer previamente de categorías.

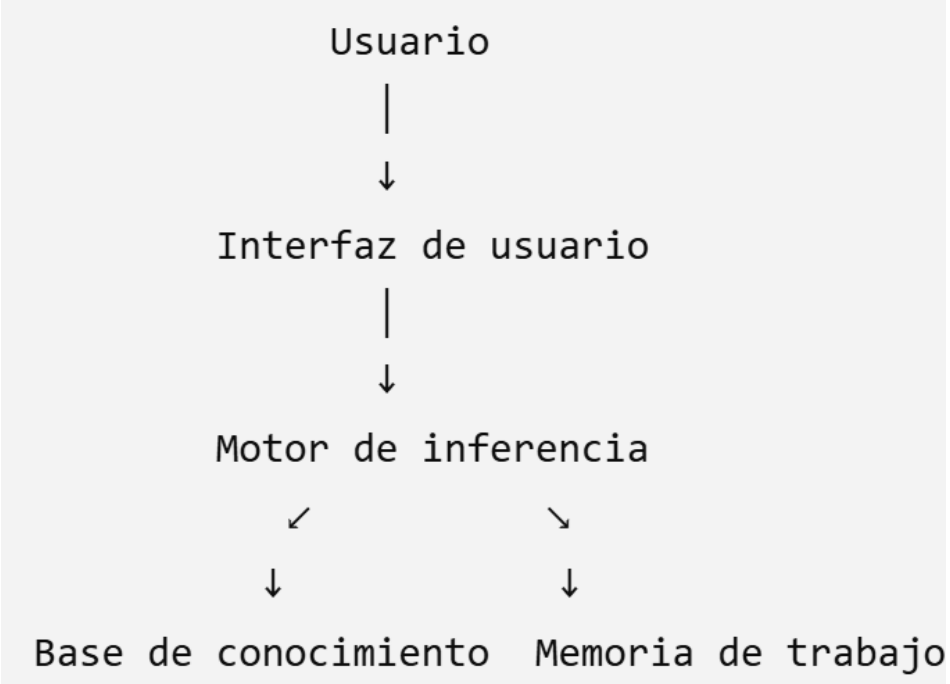
4. Sistemas basados en conocimiento

Un **Sistema Basado en Conocimiento (SBC)** es un sistema de Inteligencia Artificial que utiliza una representación explícita del conocimiento de un dominio y mecanismos de inferencia para resolver problemas o generar conclusiones.

Su característica fundamental es la separación entre:

- **conocimiento**
- **mecanismo de razonamiento**

Una arquitectura clásica puede expresarse como:



Base de conocimiento

La **base de conocimiento** contiene hechos, conceptos, relaciones, reglas y restricciones correspondientes a un dominio.

Por ejemplo:

$$Ave(Tweety)$$

$$Ave(x) \rightarrow Animal(x)$$

$$Animal(x) \rightarrow NecesitaAlimento(x)$$

El conocimiento puede representarse mediante diferentes formalismos:

- lógica proposicional
- lógica de primer orden
- reglas de producción
- redes semánticas
- frames
- ontologías
- RDF/OWL

Memoria de trabajo

La memoria de trabajo contiene los hechos relevantes para una determinada sesión de razonamiento.

Por ejemplo:

- Temperatura(Paciente, 39)
- Tos(Paciente)
- DolorGarganta(Paciente)
- El motor de inferencia utiliza estos hechos junto con las reglas de la base de conocimiento.

Motor de inferencia

El **motor de inferencia** es el componente responsable de aplicar reglas y obtener conclusiones.

```
HECHOS
  ↓
Reglas aplicables
  ↓
Nuevos hechos
  ↓
Nuevas reglas
  ↓
Conclusión
```

Dos mecanismos clásicos son:

Encadenamiento hacia adelante

- También denominado **forward chaining**.
- Parte de los hechos conocidos y aplica reglas hasta obtener nuevas conclusiones.

Ejemplo:

$$Fiebre(Paciente)$$

$$Fiebre(x) \land Tos(x) \rightarrow PosibleInfeccion(x)$$

Por tanto:

$$PosibleInfeccion(Paciente)$$

Es un enfoque **data-driven**.

Motor de inferencia

Encadenamiento hacia atrás

- El **backward chaining** parte de una hipótesis o meta y busca determinar qué hechos son necesarios para demostrarla.

Por ejemplo:

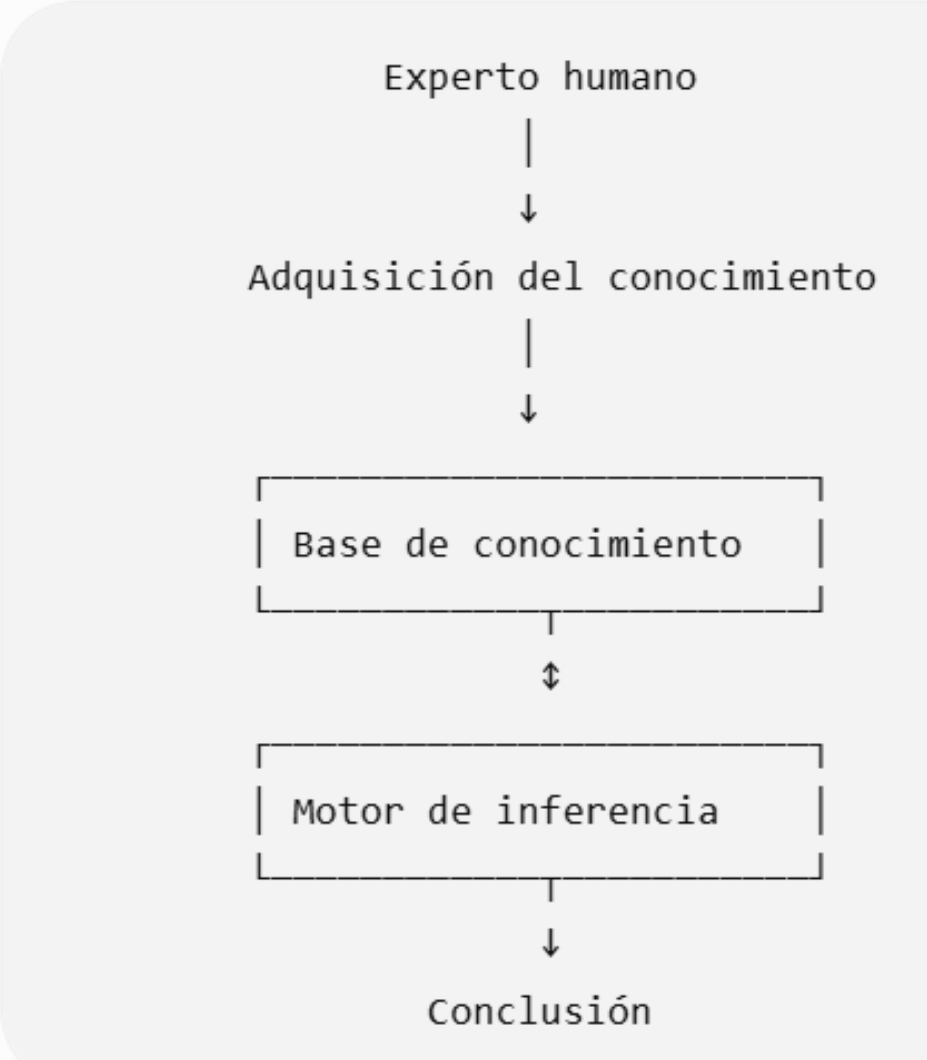
```
Meta:
PosibleInfeccion(Paciente)
        ↓
¿Qué reglas producen esta conclusión?
        ↓
Fiebre(Paciente)
Tos(Paciente)
        ↓
¿Son verdaderos estos hechos?
        ↓
Sí
        ↓
Conclusión demostrada
```

Es un enfoque **goal-driven**.

Sistemas expertos

- Los **sistemas expertos** constituyen una aplicación clásica de los sistemas basados en conocimiento.
- Su objetivo es resolver problemas en un dominio específico utilizando conocimiento especializado.

Una arquitectura tradicional incluye:



Históricamente, algunos de los sistemas expertos más conocidos fueron:

- **DENDRAL**
- **MYCIN**
- **XCON**

MYCIN, por ejemplo, utilizaba reglas para apoyar el diagnóstico de determinadas infecciones bacterianas.

Representación mediante reglas

Una representación clásica utiliza reglas de producción:

$$IF\ Condición_1 \wedge Condición_2\ THEN\ Conclusión$$

Por ejemplo:

$$IF\ temperatura > 38 \wedge tos = true$$

$$THEN\ posible_infeccion = true$$

La ventaja de esta representación es su relativa facilidad para ser interpretada por especialistas humanos.

Ventajas y limitaciones

Ventajas

- Conocimiento explícito.
- Capacidad de explicación.
- Separación entre conocimiento e inferencia.
- Posibilidad de incorporar conocimiento experto.
- Razonamiento reproducible.

Limitaciones

- Adquisición del conocimiento costosa.
- Dificultad para representar conocimiento ambiguo.
- Mantenimiento complejo cuando la base de reglas crece.
- Problemas para manejar conocimiento incompleto.
- Limitaciones frente a dominios altamente dinámicos.
- Esta última cuestión explica, en parte, por qué los sistemas modernos combinan técnicas simbólicas con **Machine Learning** y modelos probabilísticos.

5. Lenguajes y formalismos para la representación del conocimiento

- La **representación del conocimiento** estudia cómo formalizar conocimiento de manera que pueda ser almacenado, procesado e inferido computacionalmente.
- Es importante distinguir entre **lenguaje de representación** y **formalismo de representación**. Algunos enfoques son lenguajes concretos; otros son marcos formales que pueden implementarse mediante diferentes lenguajes.

Lógica proposicional

La lógica proposicional trabaja con proposiciones que pueden asumir valores de verdad.

Por ejemplo:

$$P = "El\ estudiante\ estudia"$$

$$Q = "El\ estudiante\ aprueba"$$

Una regla puede representarse:

$$P \rightarrow Q$$

Los operadores fundamentales son:

Operador	Significado
¬	Negación
∧	Conjunción
∨	Disyunción
→	Implicación
↔	Bicondicional

La principal limitación de la lógica proposicional es que no representa directamente **individuos, variables y relaciones**.

Lógica de primer orden

La **lógica de primer orden (FOL)** amplía la lógica proposicional mediante:

- constantes
- variables
- predicados
- funciones
- Cuantificadores

La lógica de primer orden proporciona una base formal para numerosos sistemas de representación del conocimiento.

Por ejemplo:

$$Humano(Socrates)$$

y:

$$\forall x(Humano(x) \rightarrow Mortal(x))$$

Permiten inferir:

$$Mortal(Socrates)$$

Los cuantificadores fundamentales son:

$$\forall$$

"para todo"

y:

$$\exists$$

"existe".

PROLOG

Prolog (*PROgramming in LOGic*) es un lenguaje de programación lógica particularmente importante en IA simbólica.

El conocimiento puede expresarse mediante hechos:

```
humano(socrates).
```

y reglas:

```
mortal(X) :- humano(X).
```

La consulta:

```
mortal(socrates).
```

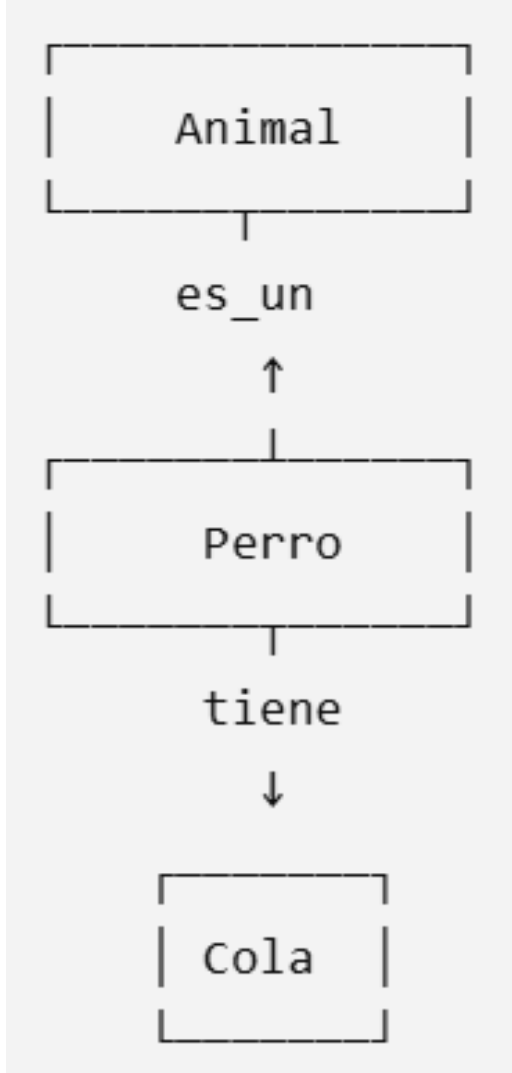
produce una respuesta afirmativa porque Prolog puede realizar inferencia lógica sobre los hechos y reglas.

Su importancia histórica radica en su estrecha relación con la **lógica de primer orden y la programación declarativa**.

Redes semánticas

- Las redes semánticas representan conocimiento mediante un **grafo etiquetado**.
- Los nodos representan entidades o conceptos y las aristas representan relaciones.

Por ejemplo:



Este formalismo constituye un antecedente conceptual de los actuales **Knowledge Graphs**.

Frames

- Los **frames**, introducidos por Marvin Minsky, representan entidades mediante estructuras que contienen atributos o *slots* y sus valores.

Por ejemplo:

```
FRAME: ESTUDIANTE

nombre: Marcela
edad: 20
carrera: Ingeniería de Software
promedio: 9.2
```

Pueden existir relaciones de herencia:

```
Persona
   ↑
Estudiante
   ↑
EstudianteIngenieria
```

Los frames resultan adecuados para representar conocimiento estructurado sobre objetos y situaciones.

Lenguaje de representación de ontologías: OWL

OWL (Web Ontology Language) es uno de los lenguajes fundamentales para representar ontologías en la Web Semántica.

Permite definir:

- clases
- individuos
- propiedades
- restricciones
- relaciones entre clases
- equivalencias
- disjunciones
- restricciones de cardinalidad

Por ejemplo:

```
Clase:
    Estudiante

Clase:
    Carrera

Propiedad:
    estudia

Dominio:
    Estudiante

Rango:
    Carrera
```

Puede establecerse:
Estudiante⊑Persona
lo que indica que:
Todo estudiante es una persona.

OWL se fundamenta en las **Description Logics**, una familia de lógicas formales diseñadas para representar conocimiento estructurado y permitir razonamiento computacional.

RDF

RDF (Resource Description Framework) representa información mediante triples:

Sujeto→Predicado→Objeto

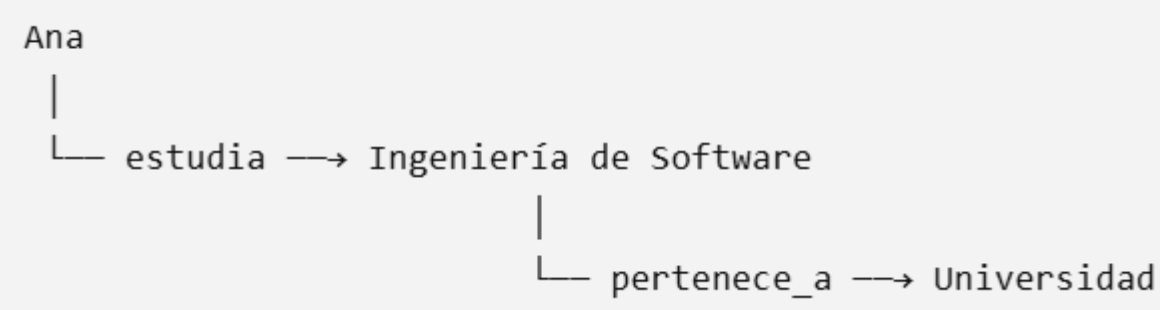
Por ejemplo:

Ana→estudia→IngenieríaSoftware

Otra afirmación:

Ingeniería Software→perteneceA→UAEM

Esto permite construir grafos de conocimiento:



RDF es particularmente importante en:

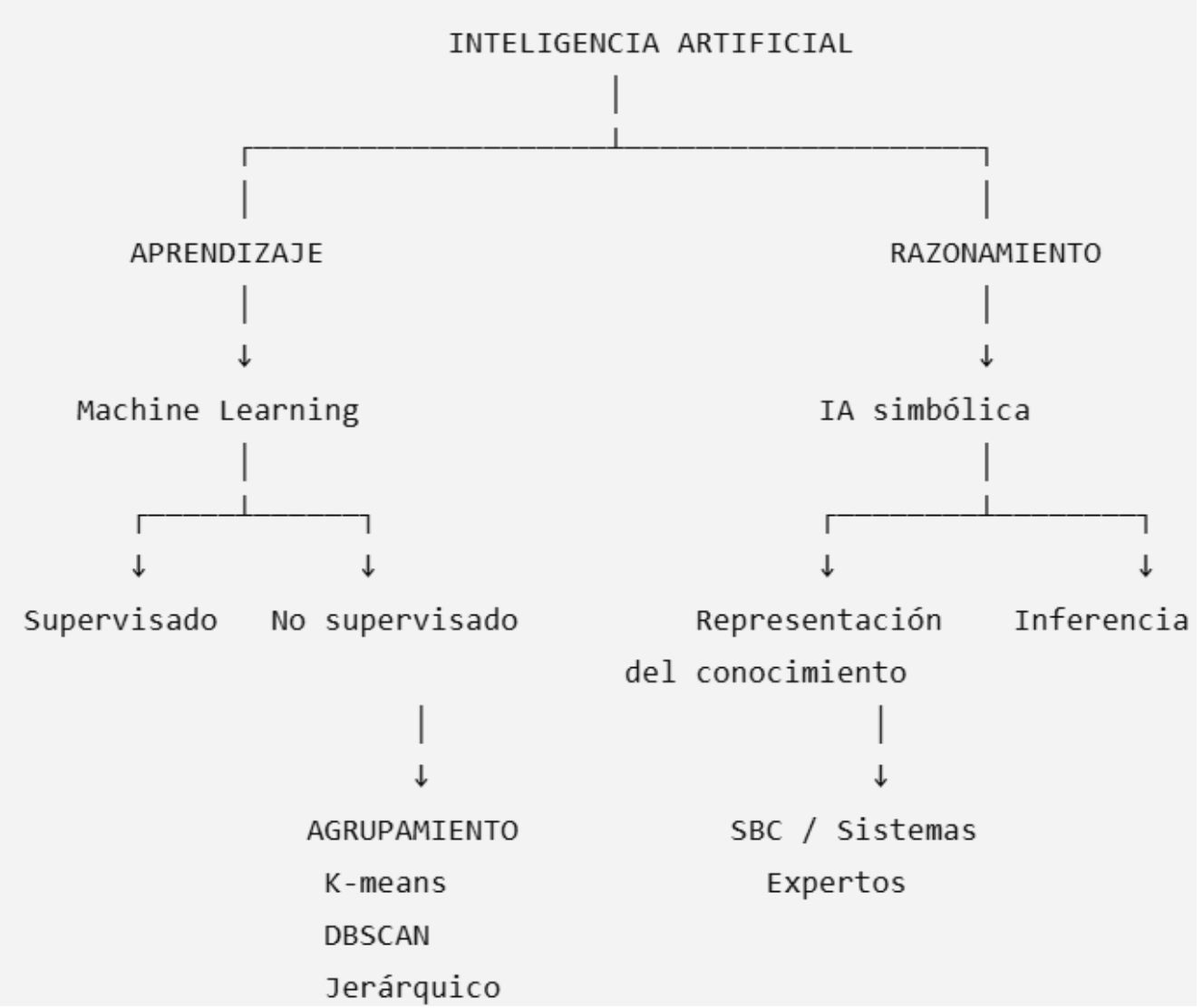
- Web Semántica
- Knowledge Graphs
- integración de datos
- interoperabilidad semántica

Comparación de formalismos

Formalismo	Elementos principales	Capacidad
Lógica proposicional	Proposiciones	Básica
Lógica de primer orden	Variables, predicados, cuantificadores	Alta
Reglas de producción	Condiciones y conclusiones	Alta para sistemas expertos
Redes semánticas	Nodos y relaciones	Alta para relaciones conceptuales
Frames	Objetos, atributos y valores	Alta para estructuras
Prolog	Hechos y reglas	Inferencia lógica
RDF	Triples	Grafos de conocimiento
OWL	Clases, propiedades y restricciones	Ontologías y razonamiento

Relación entre representación, razonamiento y aprendizaje

Los tres temas pueden integrarse dentro de una arquitectura más amplia de IA:



Esta distinción es importante porque **clustering y sistemas basados en conocimiento pertenecen a paradigmas diferentes**:

- El clustering aprende estructuras a partir de datos.
- Un SBC utiliza conocimiento explícitamente representado.
- Los sistemas modernos pueden combinar ambos enfoques.

Por ejemplo, un sistema educativo podría utilizar **K-means** para descubrir perfiles de estudiantes y posteriormente utilizar un **sistema basado en reglas** para generar recomendaciones académicas.